
Practice Assignments & Assessment Guide

Databricks Operations & Job Troubleshooting

Professional Workbook

Eight Applied Assignments in Cluster Architecture,
Workflow Orchestration, Governance & Production Reliability

Contents

1	Workspace Infrastructure & Deployment Assignment	2
1.1	Independent Coding Challenge	3
2	Asset Lifecycle Management Challenge	5
2.1	Independent Coding Challenge	6
3	Optimized Pipeline Mutation Assessment	9
3.1	Independent Coding Challenge	10
4	Enterprise Pipeline Scheduling & Production Optimization	12
4.1	Independent Coding Challenge	13
5	Advanced Time Travel & Pipeline Fault-Tolerance	16
5.1	Independent Coding Challenge	17
6	Governance Compliance & Security Controls Assessment	19
6.1	Independent Coding Challenge	20
7	Log Analysis & Infrastructure Diagnostic Auditing	23
7.1	Independent Coding Challenge	24
8	Multi-Task Recovery Protocols & Automated Repair Runs	27
8.1	Independent Coding Challenge	28
9	Enterprise Architecture Capstone Challenge — Part 1	31
9.1	Independent Coding Challenge	32
10	Enterprise Architecture Capstone Challenge — Part 2	35
10.1	Independent Coding Challenge	36

CHAPTER 1

Workspace Infrastructure & Deployment Assignment

Scenario-Based Business Problem

GlobalTrade Logistics, a multi-national supply chain conglomerate, is undergoing a massive data architecture modernization effort, migrating their legacy on-premises Hadoop clusters to a cloud-native Databricks environment. Their daily ingestion pipeline processes over 50 terabytes of unstructured telematics data, customs declarations, and warehouse inventory logs. The data engineering division consists of 45 active engineers who run iterative, highly concurrent ETL scripts and interactive exploratory data analysis (EDA). The leadership team has mandated a unified workspace that strictly segregates the control plane from the customer data plane to satisfy rigorous international compliance regulations.

Currently, the engineering team is suffering from extreme resource contention and budget overruns. Engineers have been manually spinning up massive, static clusters that remain active over the weekend, leading to a 300% spike in cloud compute costs. Furthermore, interactive notebook execution is experiencing high latency because data science workloads are competing for driver node memory alongside heavy ETL transformations. The infrastructure architects need a standardized, repeatable template for deploying multi-node development clusters that balance performance with strict cost-control measures.

As the Lead Data Architect, your mandate is to design and configure a scalable multi-node development cluster architecture that adheres to the new FinOps guidelines. You must define the exact workspace infrastructure components required to support this concurrent user base, explicitly detailing the cluster sizing, auto-scaling thresholds, and node type selections for both the driver and worker nodes. Your configuration must ensure that interactive EDA workloads do not crash the driver node, while simultaneously guaranteeing that idle compute is terminated autonomously to prevent unauthorized budget consumption.

Assignment Technical Requirements

To pass this assignment, your cluster deployment configuration must adhere to the following strict architectural constraints and configuration rules:

- **Compute Paradigm:** The cluster must be configured as an All-Purpose Cluster meant for interactive development, not a Job Cluster.
- **Driver Node Sizing:** The driver node must be explicitly separated from worker node types. It must utilize a memory-optimized instance (e.g., AWS `r5.2xlarge` or Azure `Standard_E8s_v3`) to support high-concurrency notebook state management and meta-data operations.

- **Worker Node Sizing:** Worker nodes must be compute-optimized (e.g., AWS c5.2xlarge or Azure Standard_F8s_v2) to handle heavy Spark task execution.
- **Auto-Scaling Constraints:** Static provisioning is strictly prohibited. The cluster must have auto-scaling enabled with a minimum floor of 2 worker nodes and a maximum ceiling of 12 worker nodes.
- **Cost Management Policy:** Auto-termination must be enforced. The cluster must automatically spin down after exactly 45 minutes of inactivity.
- **Databricks Runtime (DBR):** Must specify a Long Term Support (LTS) version, specifically DBR 13.3 LTS (or equivalent syntax 13.3.x-scala2.12).
- **Security & Access:** Cluster access control must be enabled, granting CAN_ATTACH_TO permissions only to the `data_engineering_group`.

1.1 Independent Coding Challenge

Using the Databricks REST API 2.0 structure for cluster creation, complete the following JSON configuration payload. You must replace all [TODO] placeholders with the precise configuration values that satisfy the architectural constraints outlined above.

```

1  {
2    "cluster_name": "GlobalTrade_Dev_Cluster_01",
3    "spark_version": "[TODO: Specify the exact DBR LTS version string
4      ]",
5    "node_type_id": "[TODO: Specify the worker node instance type]",
6    "driver_node_type_id": "[TODO: Specify the driver node instance
7      type]",
8    "autotermination_minutes": [TODO: Insert integer for inactivity
9      timeout],
10   "autoscale": {
11     "min_workers": [TODO: Insert minimum node count],
12     "max_workers": [TODO: Insert maximum node count]
13   },
14   "spark_conf": {
15     "spark.databricks.cluster.profile": "serverless",
16     "spark.databricks.repl.allowedLanguages": "sql,python,r,scala"
17   },
18   "custom_tags": {
19     "Department": "DataEngineering",
20     "CostCenter": "CC-7784"
21   },
22   "access_control_list": [
23     {
24       "group_name": "[TODO: Insert target group name]",
25       "permission_level": "[TODO: Insert explicit permission string
26         ]"
27     }
28   ]
29 }
```

```
24 ]
25 }
```

Grading Rubric & Solution Verification

Maximum Score: 100 Points (Minimum Passing Score: 85)

- **Runtime Version Accuracy (15 points):** Pass/Fail. The `spark_version` must be explicitly defined as an LTS version (e.g., "13.3.x-scala2.12"). Use of a non-LTS or ML-specific runtime results in zero points for this section.
- **Driver vs. Worker Asymmetry (25 points):** The `driver_node_type_id` must be a memory-optimized class (15 points), and the `node_type_id` must be a compute-optimized class (10 points).
- **Auto-scaling Configuration (25 points):**
 - `min_workers` strictly set to 2 (10 points).
 - `max_workers` strictly set to 12 (10 points).
 - Hardcoding a `num_workers` field instead of using the `autoscale` block will result in an automatic failure for the entire assignment.
- **FinOps Compliance (20 points):** `autotermination_minutes` must be exactly 45. Any value higher or missing results in a deduction of 20 points.
- **Security & ACLs (15 points):** The `access_control_list` must specify `"group_name": "data_engineering_group"` and `"permission_level": "CAN_ATTACH_TO"`.

Verification Query Structure

Upon submission, the examiner will dry-run the JSON payload against the Databricks `api/2.0/clusters/create` endpoint. The cluster must achieve a PENDING to RUNNING state transition within 4 minutes without throwing capacity or permission errors.

CHAPTER 2

Asset Lifecycle Management Challenge

Scenario-Based Business Problem

FinSecure Analytics provides real-time and batch risk modeling for tier-one investment banks. Their core product is a nightly Value at Risk (VaR) calculation that processes over 10 million individual portfolios against historical market fluctuations. The data processing pipeline involves a complex dependency graph of notebooks: a data extraction notebook, a Monte Carlo simulation notebook, and a final reporting notebook that writes to a Gold-level Delta table. Currently, the operations team triggers these notebooks manually or via a legacy cron scheduler attached to a persistently running All-Purpose cluster.

The operational overhead has become unsustainable. Because the workflows are executed on interactive clusters, the cluster lifecycle is completely decoupled from the job execution lifecycle. If a notebook fails halfway through the Monte Carlo simulation, the cluster remains active, burning compute credits for hours. Furthermore, variables and temporary views are bleeding across notebook execution scopes because the `SparkSession` is shared and not properly isolated between runs. The operations team is struggling to pass dynamic execution dates down the pipeline, relying on fragile hardcoded variables that require manual daily updates.

Your objective is to re-architect this workflow into a robust, automated pipeline. You must eliminate the use of All-Purpose clusters for this production workflow, orchestrating the execution using ephemeral Job clusters that strictly bind the compute lifecycle to the workload execution. Furthermore, you are required to implement proper notebook variable scoping using Databricks Widgets, allowing the parent orchestration script to dynamically pass the `business_date` parameter to the child notebooks, ensuring state isolation and idempotent execution.

Assignment Technical Requirements

To pass this assignment, your cluster orchestration and notebook parameterization code must adhere to the following:

- **Workflow Orchestration:** You must use Databricks Utilities (`dbutils.notebook.run`) to orchestrate the execution of the child notebook from a parent master notebook.
- **Dynamic Parameter Passing:** The parent notebook must pass a `business_date` parameter (format: YYYY-MM-DD) and a `risk_threshold` parameter (float) to the child notebook.
- **Widget Implementation:** The child notebook must utilize `dbutils.widgets` to define, retrieve, and validate these incoming parameters.

- **Scope Isolation:** You must ensure that the child notebook executes in a separate context. (By definition, `dbutils.notebook.run` achieves this, but you must capture the return value of the child notebook to verify success).
- **Exception Handling:** The orchestration must include a timeout constraint. If the child notebook (`monte_carlo_sim`) runs for more than 3600 seconds, it must be forcefully terminated, and the parent notebook must catch the `TimeoutError` and log a failure metric.

2.1 Independent Coding Challenge

Part A: Child Notebook (`monte_carlo_sim`) Parameterization

Complete the boilerplate PySpark code to properly instantiate widgets and retrieve their values.

```

1  # monte_carlo_sim notebook
2  # TODO: Define a text widget for 'business_date' with a default
   value of '1970-01-01'
3  # TODO: Define a dropdown widget for 'risk_threshold' with choices
   ['0.01', '0.05', '0.10'] and default '0.05'
4
5  # TODO: Retrieve the values from the widgets into local Python
   variables
6  biz_date = ""
7  risk_val = ""
8
9  print(f"Executing Monte Carlo for date: {biz_date} with threshold {
   risk_val}")
10
11 # Simulate workload
12 import time
13 time.sleep(10)
14
15 # TODO: Exit the notebook successfully, returning a string
   confirmation "SUCCESS_VAR_CALC"
```

Part B: Parent Orchestration Notebook

Complete the boilerplate code to execute the child notebook with the strict timeout and parameter requirements.

```

1  # master_orchestrator notebook
2  from datetime import datetime
3
4  current_run_date = datetime.now().strftime('%Y-%m-%d')
5  target_notebook_path = "/Shared/RiskModels/monte_carlo_sim"
6
7  try:
8      # TODO: Use dbutils.notebook.run to execute the target notebook.
```

```

9      # Set the timeout to exactly 3600 seconds.
10     # Pass the 'business_date' as current_run_date and '
        risk_threshold' as '0.05'
11
12     result = "" # Implement execution here
13
14     if result == "SUCCESS_VAR_CALC":
15         print("Child notebook executed successfully in isolated
            scope.")
16     else:
17         print(f"Unexpected return state: {result}")
18
19 except Exception as e:
20     # TODO: Catch the specific notebook execution exception and
        print a formatted alert
21     print("FATAL ALERT: Notebook execution failed or timed out.")
22     raise e

```

Grading Rubric & Solution Verification

Maximum Score: 100 Points (Minimum Passing Score: 85)

- **Widget Definition & Retrieval (30 points):**

- Correct use of `dbutils.widgets.text()` and `dbutils.widgets.dropdown()` (15 points).
- Correct retrieval using `dbutils.widgets.get()` (15 points). Hardcoding variables overriding the widgets will result in a zero for this section.

- **Notebook Exit State (15 points):** The child notebook must explicitly use `dbutils.notebook.exit("SUCCESS_VAR_CALC")`. Standard return statements are invalid in this context.

- **Orchestration API Syntax (35 points):**

- Correct invocation of `dbutils.notebook.run(target_notebook_path, 3600, {"business_date": current_run_date, "risk_threshold": "0.05"})` (25 points).
- Strict adherence to the 3600-second timeout parameter (10 points).

- **Error Handling (20 points):** The parent notebook must properly implement a try-except block capable of catching Databricks-specific `WorkflowException` or standard Python `Exception` to prevent silent failures in the ETL chain.

Verification Query Structure

The examiner will run the parent notebook. The verification script will parse the Spark logs to ensure that the execution of `/Shared/RiskModels/monte_carlo_sim` spun up a

distinct Job execution context (verifiable via the Job Run ID) rather than executing as a cell command in the interactive session.

CHAPTER 3

Optimized Pipeline Mutation Assessment

Scenario-Based Business Problem

MediStream Health operates an international network of IoT medical devices that stream patient heart rate, blood oxygen levels, and body temperature to a centralized data lake. This telemetry data amounts to 4.5 billion rows per day. At the end of each day, the engineering team runs a batch upsert process to update the `patient_vitals_gold` Delta table. If a reading for a specific `patient_id` and `event_timestamp` already exists (perhaps due to a late-arriving packet correction), the record must be updated; otherwise, the new reading is inserted.

Currently, this critical pipeline is on the verge of total collapse. The team is running the MERGE pipeline using an All-Purpose cluster, leading to excessive DBU costs (often 3x higher than Job clusters). Worse, the Spark job frequently stalls for 6 hours before failing with OutOfMemory (OOM) errors. An analysis of the Spark UI reveals that the `MERGE INTO` statement is performing a full table scan across 4 years of historical data for every single micro-batch. The Delta table is massive, unoptimized, and lacks any form of data skipping or strategic partitioning.

Your mission is to completely refactor this batch mutation pipeline. You must transition the execution profile from an interactive compute model to a dedicated Job cluster architecture for optimal compute pricing. More importantly, you must rewrite the Spark SQL `MERGE` logic. You are required to enforce partition pruning dynamically during the merge operation and implement a Z-ORDER clustering strategy to colocate similar patient IDs, thereby drastically reducing the volume of files read during the join phase of the upsert.

Assignment Technical Requirements

To pass this performance tuning assessment, your implementation must satisfy the following architectural and data constraints:

- **Compute Constraint:** You must theoretically target a Job Cluster (automated compute) rather than an All-Purpose cluster, optimizing for throughput over interactive latency.
- **Delta Lake Layout:** The target table `patient_vitals_gold` is partitioned by `event_date` (DateType). You must leverage this partition column in your MERGE condition to prevent full table scans.
- **Dynamic Partition Pruning:** The source dataframe (the daily updates) contains data for only the last 3 days. Your MERGE statement must dynamically restrict the target table search space to only those specific 3 days based on the source data.

- **Z-Ordering Execution:** After the merge completes, you must execute an `OPTIMIZE` command on the table, explicitly applying `ZORDER BY (patient_id, device_id)` to optimize future point lookups.
- **Language Requirement:** You must use standard Databricks Spark SQL for the `MERGE` and `OPTIMIZE` commands.

3.1 Independent Coding Challenge

You are provided with a temporary view named `source_telemetry_updates` containing today's upsert data. Write the highly optimized SQL script to perform the `MERGE` and subsequent optimization. Replace the `[TODO]` sections with valid, performant Spark SQL.

```

1  -- Part A: Optimized MERGE Statement
2  -- Target table: main.healthcare.patient_vitals_gold
3  -- Source view: source_telemetry_updates
4  -- Join keys: patient_id, event_timestamp
5
6  MERGE INTO main.healthcare.patient_vitals_gold AS target
7  USING source_telemetry_updates AS source
8  ON target.patient_id = source.patient_id
9     AND target.event_timestamp = source.event_timestamp
10     -- TODO: Implement Dynamic Partition Pruning here to limit the
11         target table scan
12     -- Hint: Restrict target.event_date to only match the event_dates
13         present in the source view
14     [TODO: INSERT PARTITION PRUNING LOGIC]
15
16 WHEN MATCHED THEN
17     -- TODO: Implement the update logic for 'heart_rate' and '
18         oxygen_level '
19     [TODO: INSERT UPDATE LOGIC]
20
21 WHEN NOT MATCHED THEN
22     -- TODO: Implement the insert logic for all columns
23     [TODO: INSERT INSERT LOGIC];
24
25 -- Part B: Data Layout Optimization
26 -- TODO: Write the command to optimize the target table and Z-Order
27     by patient_id and device_id
28 [TODO: INSERT OPTIMIZE AND ZORDER LOGIC]

```

Grading Rubric & Solution Verification

Maximum Score: 100 Points (Minimum Passing Score: 90)

- **Compute Model Knowledge (10 points):** The candidate accurately defines (in code comments or preceding configuration) that Job Clusters charge lower DBU rates and isolate workloads, fulfilling the FinOps requirement.
- **Dynamic Partition Pruning Implementation (40 points):**
 - The ON clause must include a condition that filters the target table's partition column (`event_date`).
 - Sample accepted solution: `AND target.event_date >= (SELECT MIN(event_date) FROM source_telemetry_updates)` or equivalent explicit date filtering.
 - Failure to include partition pruning in a table this size results in an automatic failure (0 points for this section), as it will guarantee an OOM error in production.
- **Merge Clauses (20 points):**
 - Correct syntax for `WHEN MATCHED UPDATE SET ...` (10 points).
 - Correct syntax for `WHEN NOT MATCHED INSERT *` or explicit column mapping (10 points).
- **Z-Ordering Optimization (30 points):**
 - Must use the exact syntax: `OPTIMIZE main.healthcare.patient_vitals_gold ZORDER BY (patient_id, device_id)`.
 - Any syntax error or omitting one of the columns results in a deduction of 15 points.

Verification Query Structure

The examiner will run the `EXPLAIN` plan on the submitted `MERGE` statement. The query plan must explicitly show `PartitionFilters: [isnotnull(event_date#...), (event_date#... >= ...)]` in the physical plan for the target table scan. If the physical plan indicates a full scan of all historical partitions, the solution is immediately rejected. Execution time must drop from the baseline of >6 hours to under 12 minutes.

CHAPTER 4

Enterprise Pipeline Scheduling & Production Optimization

Scenario-Based Business Problem

NexusCommerce, an international e-commerce conglomerate, processes millions of daily transactions, clickstream events, and inventory updates. Their central data engineering team is actively migrating their legacy orchestration framework—previously managed by external Apache Airflow instances that triggered Databricks notebooks—into native Databricks Workflows. The legacy architecture incurred excessive network latency and API polling overhead, while simultaneously inflating costs due to the provisioning of distinct clusters for every isolated task. The Chief Data Officer has mandated a transition to a unified Medallion Architecture (Bronze, Silver, Gold) orchestrated entirely within a single Databricks Multi-Task Job.

The pipeline, named `Daily_Retail_ETL`, is highly complex and structurally interdependent. It consists of four distinct processing stages. First, the `Ingest_Bronze` task pulls raw JSON clickstream data from an AWS S3 bucket. Second, the `Clean_Silver` task deduplicates the data and enforces schema validation. Third, the `Aggregate_Gold` task joins the Silver clickstream data with dimensional user tables to generate enriched marketing metrics. Finally, a parallel task, `Update_Dashboard`, refreshes a Databricks SQL endpoint based on the Gold layer. Currently, engineers are struggling with dependency mapping—tasks are running out of order or concurrently when they shouldn't, causing data corruption. Furthermore, they are spinning up separate Job Clusters for each task, wasting 10–15 minutes on cluster bootstrapping per step and driving compute costs beyond acceptable budget limits.

Your objective as the Lead Data Engineer is to architect a Databricks Multi-Task Job configuration that strictly defines the execution dependencies (a Directed Acyclic Graph – DAG) and drastically optimizes cluster utilization. You must enforce the use of a single shared Job Cluster across all tasks within the workflow, ensuring that the compute engine is bootstrapped only once at the beginning of the pipeline and reused across the Bronze, Silver, Gold, and Reporting tasks.

Assignment Technical Requirements

To pass this pipeline optimization assessment, your Databricks Workflow JSON definition must adhere to the following strict architectural constraints:

- **Compute Optimization (Shared Job Cluster):** You must define a single `job_cluster` at the root of the JSON payload. Individual tasks must reference this

shared cluster using the `job_cluster_key` rather than defining their own `new_cluster` or utilizing an `existing_cluster_id` (All-Purpose cluster).

- **Dependency Mapping (DAG Construction):** You must construct a strict execution order using the `depends_on` array within the task definitions:
 - `Clean_Silver` must wait for `Ingest_Bronze` to complete.
 - `Aggregate_Gold` must wait for `Clean_Silver` to complete.
 - `Update_Dashboard` must wait for `Aggregate_Gold` to complete.
- **Delta Lake Fundamentals (Task Types):** The first three tasks must be configured as `notebook_task` objects. The final task, `Update_Dashboard`, must be configured as a `sql_task` that triggers an existing query or dashboard update.
- **Failure & Retry Policies:** The `Ingest_Bronze` task is prone to temporary S3 network timeouts. You must configure a retry policy for this specific task, allowing up to 3 retries with a 300000-millisecond (5-minute) delay between attempts.
- **Timeouts:** The entire multi-task job must have a hard timeout of 120 minutes (7200 seconds) to prevent zombie processes during massive data spikes.

4.1 Independent Coding Challenge

Using the Databricks Jobs API 2.1 JSON structure, complete the workflow configuration payload below. You must replace all [TODO] placeholders with the precise configuration values, dependency mappings, and cluster references to satisfy the architectural constraints.

```
1 {
2   "name": "Daily_Retail_ETL_Optimized",
3   "timeout_seconds": [TODO: Insert Job-level timeout in seconds],
4   "job_clusters": [
5     {
6       "job_cluster_key": "shared_etl_cluster",
7       "new_cluster": {
8         "spark_version": "13.3.x-scala2.12",
9         "node_type_id": "r5.4xlarge",
10        "num_workers": 8
11      }
12    }
13  ],
14  "tasks": [
15    {
16      "task_key": "Ingest_Bronze",
17      "job_cluster_key": "[TODO: Reference the shared cluster key]",
18      "notebook_task": {
19        "notebook_path": "/Production/ETL/01_Ingest_Bronze"
20      },
21      "max_retries": [TODO: Insert retry limit],
```

```

22     "min_retry_interval_millis": [TODO: Insert retry delay in
23         milliseconds]
24 },
25 {
26     "task_key": "Clean_Silver",
27     "depends_on": [
28         {
29             "task_key": "[TODO: Insert upstream dependency]"
30         }
31     ],
32     "job_cluster_key": "[TODO: Reference the shared cluster key]",
33     "notebook_task": {
34         "notebook_path": "/Production/ETL/02_Clean_Silver"
35     },
36     {
37         "task_key": "Aggregate_Gold",
38         "depends_on": [
39             {
40                 "task_key": "[TODO: Insert upstream dependency]"
41             }
42         ],
43         "job_cluster_key": "[TODO: Reference the shared cluster key]",
44         "notebook_task": {
45             "notebook_path": "/Production/ETL/03_Aggregate_Gold"
46         }
47     },
48     {
49         "task_key": "Update_Dashboard",
50         "depends_on": [
51             {
52                 "task_key": "[TODO: Insert upstream dependency]"
53             }
54         ],
55         "job_cluster_key": "[TODO: Reference the shared cluster key]",
56         "sql_task": {
57             "query": {
58                 "query_id": "8a7b6c5d-4e3f-2g1h-0i9j"
59             }
60         }
61     }
62 ]
63 }

```

Grading Rubric & Solution Verification

Maximum Score: 100 Points (Minimum Passing Score: 85)

- **Shared Compute Validation (30 points):** All four tasks must explicitly contain "job_cluster_key": "shared_etl_cluster". Using "new_cluster" inside any indi-

vidual task definition will result in an automatic failure (0 points) as it violates the core FinOps directive.

- **DAG Dependency Mapping (35 points):**

- `Clean_Silver` correctly depends on `Ingest_Bronze` (10 points).
- `Aggregate_Gold` correctly depends on `Clean_Silver` (10 points).
- `Update_Dashboard` correctly depends on `Aggregate_Gold` (15 points).

- **Retry & Fault Tolerance Configuration (20 points):** `Ingest_Bronze` must possess `"max_retries": 3` (10 points) and `"min_retry_interval_millis": 300000` (10 points).

- **Global Timeout Enforcement (15 points):** The root payload must have `"timeout_seconds": 7200`.

Verification Query Structure

Upon submission, the examiner will inject the JSON payload into the `api/2.1/jobs/create` endpoint. The platform's DAG visualization engine must successfully render a linear path without circular dependencies. A dry-run execution must reflect exactly 1 cluster boot sequence in the cluster activity logs.

CHAPTER 5

Advanced Time Travel & Pipeline Fault-Tolerance

Scenario-Based Business Problem

PayStream Financials operates a high-frequency trading platform where millisecond accuracy is critical. Their core asset is the `transactions_gold` Delta table, which serves as the source of truth for all regulatory reporting and intraday risk analytics. At 2:15 PM EST during a critical trading window, a junior data engineer attempting to run an ad-hoc backfill operation executed a devastating SQL `UPDATE` statement. They forgot to include the `WHERE` clause, effectively overwriting the `transaction_status` column to "PENDING" for 4.2 billion historical records across the entire Delta table. Downstream dashboards instantly failed, and automated risk models triggered false alerts to regulatory agencies.

The operations team scrambled to cancel the rogue workflow, leaving the pipeline in a fractured, inconsistent state. Because the underlying storage format is Delta Lake, the transaction log has recorded this mass mutation, but the physical Parquet files from previous states still exist in cloud storage (assuming the `VACUUM` command has not yet been executed). Time is of the essence; every minute of downtime costs the firm tens of thousands of dollars in SLA penalties. Standard restore-from-backup procedures would take 18 hours to copy data back from cold storage, which is unacceptable.

Your critical mandate is to leverage Databricks Delta Lake Time Travel capabilities to perform an instantaneous, zero-copy restoration of the corrupted table. You must interrogate the Delta transaction log to identify the precise version of the table prior to the rogue `UPDATE`, simulate a non-destructive verification of the historical state, and seamlessly roll the production table back to its uncorrupted state. Furthermore, you must ensure that subsequent read-streams configured on this table do not crash due to the sudden historical reversion.

Assignment Technical Requirements

To successfully mitigate this data disaster, your SQL recovery scripts must strictly follow these fault-tolerance procedures:

- **Log Auditing:** You must first use Databricks SQL to interrogate the Delta transaction history of `main.finance.transactions_gold` to identify the version number of the rogue `UPDATE` operation.
- **Time Travel Verification:** Before committing to a permanent restore, you must write a verification query that reads from the specific uncorrupted version of the table to prove

the data is intact (e.g., verifying that `transaction_status` contains values other than "PENDING").

- **State Restoration:** You must execute the native Delta Lake `RESTORE` command to definitively rollback the table to the identified safe version.
- **Workflow Recovery Awareness:** Because the table is being restored, downstream Structured Streaming pipelines reading from this table will throw an `OffsetMismatchException`. You must provide the PySpark streaming configuration flag required to allow the stream to gracefully ignore the missing offsets and restart from the new table state.

5.1 Independent Coding Challenge

Complete the following disaster recovery SQL/Python script. Replace the [TODO] placeholders with the exact commands necessary to audit the log, verify the historical data, restore the table, and configure the downstream recovery stream.

```
1  -- Part A: Delta Log Auditing & Verification
2  -- TODO: Write the command to view the transaction history of the
      target table
3  [TODO: INSERT HISTORY AUDIT COMMAND] main.finance.transactions_gold;
4
5  -- Assume the history reveals that Version 84 is the rogue UPDATE,
      and Version 83 is the last known good state.
6
7  -- TODO: Write a SELECT statement using Time Travel syntax to count
      the distinct 'transaction_status' values specifically from
      Version 83.
8  SELECT transaction_status, COUNT(*)
9  FROM main.finance.transactions_gold [TODO: INSERT TIME TRAVEL
      VERSION SYNTAX]
10 GROUP BY transaction_status;
11
12 -- Part B: Executing the Rollback
13 -- TODO: Write the command to completely restore the table to
      Version 83
14 [TODO: INSERT RESTORE COMMAND] main.finance.transactions_gold [TODO:
      INSERT VERSION TARGET];
```

```
1  # Part C: Pipeline Fault-Tolerance (Structured Streaming Recovery)
2  # The downstream risk application reads from transactions_gold.
3  # TODO: Add the specific option to the readStream configuration that
      allows it to
4  # bypass the OffsetMismatch exception caused by the RESTORE command.
5
6  df_stream = (spark.readStream
7    .format("delta")
```

```

8      # [TODO: Insert the required boolean option to ignore changes/
      deletes or handle offsets]
9      .option("[TODO: CONFIGURATION_KEY]", "[TODO: CONFIGURATION_VALUE]"
      )
10     .table("main.finance.transactions_gold")
11 )

```

Grading Rubric & Solution Verification

Maximum Score: 100 Points (Minimum Passing Score: 90)

- **Log Interrogation (20 points):** Candidate must use the exact syntax `DESCRIBE HISTORY main.finance.transactions_gold;` (20 points).
- **Time Travel Syntax (30 points):** The `SELECT` query must explicitly append `VERSION AS OF 83` or `@v83` immediately following the table name (30 points). Use of standard SQL `AS OF` without Delta specifics is heavily scrutinized.
- **State Restoration (30 points):**
 - Must use the `RESTORE TABLE main.finance.transactions_gold TO VERSION AS OF 83;` command.
 - Using `INSERT OVERWRITE` combined with a time-travel select is acceptable but sub-optimal; doing so yields a maximum of 15 points out of 30 due to the unnecessary compute overhead compared to a metadata-only `RESTORE`.
- **Streaming Fault-Tolerance (20 points):** The downstream stream must include `.option("ignoreChanges", "true")` or, in extreme cases of full table recreation, `.option("startingVersion", "latest")` combined with a checkpoint reset. (20 points for correctly identifying `ignoreChanges` or `ignoreDeletes`).

Verification Query Structure

The examiner framework will intentionally corrupt a test table in a sandbox environment. The submitted `RESTORE` script will be executed. The grading engine will then assert that the table metadata version is incremented, but the physical pointers refer exactly to the Parquet files from v83. The streaming query will be initiated to ensure it does not immediately terminate with a `java.lang.IllegalStateException: Offset mismatch`.

CHAPTER 6

Governance Compliance & Security Controls Assessment

Scenario-Based Business Problem

HealthCorp International operates a federated data lakehouse architecture spanning multiple cloud regions. With upcoming HIPAA compliance audits and the rollout of the General Data Protection Regulation (GDPR) in their European sectors, the Chief Information Security Officer (CISO) has mandated an immediate overhaul of Databricks workspace access controls. Previously, HealthCorp utilized Hive Metastore (HMS) with table-level Access Control Lists (ACLs), which proved impossible to manage at scale. Engineers were frequently granted blanket **SELECT** privileges across massive schemas, exposing highly sensitive Protected Health Information (PHI)—such as patient names, Social Security Numbers, and diagnostic codes—to unauthorized data analysts and offshore development teams.

To resolve this, HealthCorp has successfully activated Unity Catalog across their workspaces. They have created a highly isolated catalog named `healthcare_prod`, containing a schema named `phi_records`. Within this schema sits the critical `patient_diagnostics` Delta table. The compliance framework dictates strict Role-Based Access Control (RBAC):

1. The `compliance_auditors` group must have full read access to this specific table to verify data integrity and regulatory adherence.
2. The `data_engineers` group must be able to view the schema and the table metadata to orchestrate pipelines, but they are strictly prohibited from viewing the raw data within `patient_diagnostics`.
3. The `offshore_analysts` group must be explicitly barred from even knowing the `phi_records` schema exists.

Your assignment as the Lead Security Architect is to define the exact Unity Catalog **GRANT** and **REVOKE** access paths. You must correctly navigate the three-tier Unity Catalog namespace (Catalog → Schema → Table) to apply the principle of least privilege. Furthermore, you must provide the SQL audit commands necessary to query the system's raw compute logs to prove to the CISO who accessed the table during the last 24 hours.

Assignment Technical Requirements

To pass this governance and security assessment, your SQL permission definitions must adhere to the following rules:

- **Three-Tier Namespace Enforcement:** All GRANT and REVOKE statements must utilize the fully qualified Unity Catalog notation (`catalog.schema.table`).
- **Hierarchical Permissions:** You must understand that Unity Catalog requires USE CATALOG and USE SCHEMA privileges on parent objects before any table-level privileges (like SELECT) can be exercised by a principal.
- **Targeted Grants (Auditors):** The `compliance_auditors` group must be granted the exact chain of permissions required to execute a SELECT statement on `patient_diagnostics`.
- **Restricted Metadata Access (Engineers):** The `data_engineers` group must be granted the ability to traverse the catalog and schema, but you must ensure they do not possess SELECT or MODIFY rights on the PHI table.
- **System Auditing:** You must write a query against the Unity Catalog `system.access.audit` table to retrieve all read events (`action_name = 'SELECT'`) targeting the `patient_diagnostics` table over the last 24 hours.

6.1 Independent Coding Challenge

Write the comprehensive SQL security script to enforce the CISO's directives. Replace the [TODO] sections with valid Unity Catalog SQL syntax.

```

1  -- Part A: Provisioning Access for Compliance Auditors
2  -- TODO: Grant the necessary parent-level catalog privilege
3  [TODO: INSERT GRANT COMMAND] ON CATALOG healthcare_prod TO '
    compliance_auditors';
4
5  -- TODO: Grant the necessary parent-level schema privilege
6  [TODO: INSERT GRANT COMMAND] ON SCHEMA healthcare_prod.phi_records
    TO 'compliance_auditors';
7
8  -- TODO: Grant the data-level read privilege
9  [TODO: INSERT GRANT COMMAND] ON TABLE healthcare_prod.phi_records.
    patient_diagnostics TO 'compliance_auditors';
10
11
12 -- Part B: Provisioning Restricted Access for Data Engineers
13 -- TODO: Grant catalog traversal rights
14 [TODO: INSERT GRANT COMMAND] ON CATALOG healthcare_prod TO '
    data_engineers';
15
16 -- TODO: Grant schema traversal rights
17 [TODO: INSERT GRANT COMMAND] ON SCHEMA healthcare_prod.phi_records
    TO 'data_engineers';
18
19 -- Do NOT grant SELECT on the table.
```

```

20 -- TODO: Explicitly ensure they cannot read the table (if inherited
    from a broader group, force a deny/revoke)
21 -- Note: Unity Catalog denies by default, but write the explicit
    REVOKE for safety.
22 [TODO: INSERT REVOKE COMMAND] ON TABLE healthcare_prod.phi_records.
    patient_diagnostics FROM 'data_engineers';
23
24
25 -- Part C: Auditing Access Logs via System Tables
26 -- TODO: Query the UC system audit log to find who queried this
    table in the last day.
27 SELECT
28     user_identity.email,
29     event_time,
30     action_name
31 FROM
32     [TODO: INSERT SYSTEM AUDIT TABLE PATH]
33 WHERE
34     action_name = 'SELECT'
35     AND resource_type = 'TABLE'
36     AND request_params.table_full_name = 'healthcare_prod.
        phi_records.patient_diagnostics'
37 -- TODO: Add a timestamp filter for the last 24 hours using
    native Databricks SQL functions
38 AND [TODO: INSERT TIMESTAMP FILTER LOGIC];

```

Grading Rubric & Solution Verification

Maximum Score: 100 Points (Minimum Passing Score: 85)

- **Hierarchical Traversal Privileges (30 points):**

- Must explicitly GRANT USE CATALOG ON CATALOG healthcare_prod (15 points).
- Must explicitly GRANT USE SCHEMA ON SCHEMA healthcare_prod.phi_records (15 points). Using GRANT ALL PRIVILEGES results in an automatic failure (0 points for the entire exam) due to severe security violations.

- **Targeted Table Grants (20 points):** Must explicitly GRANT SELECT ON TABLE healthcare_prod.phi_records.patient_diagnostics TO `compliance\_auditors`.

- **Explicit Denials/Revocations (20 points):** Must explicitly write REVOKE SELECT ON TABLE healthcare_prod.phi_records.patient_diagnostics FROM `data\_engineers` to satisfy the defensive programming requirement against accidental group inheritance.

- **System Table Auditing (30 points):**

- Must correctly target the Unity Catalog audit log: system.access.audit (15 points).

- Must implement a valid time filter, such as `event_time >= current_timestamp()`
- `INTERVAL 1 DAY` or `event_date >= current_date() - INTERVAL 1 DAY` (15 points).

Verification Query Structure

The grading environment will deploy these exact SQL statements to an active Unity Catalog metastore. The examiner script will then assume the identity of a user in the `data_engineers` group and attempt to execute `SELECT * FROM healthcare_prod.phi_records.patient_diagnostics`. The system must throw an `AnalysisException` stating *User does not have SELECT on Table*. Conversely, executing the same command as a `compliance_auditor` must succeed and return a dataset.

CHAPTER 7

Log Analysis & Infrastructure Diagnostic Auditing

Scenario-Based Business Problem

AeroComm Telecommunications operates a massive cellular network grid, ingesting over 10 billion network telemetry events daily to monitor cell tower health and predictive maintenance needs. Their primary ETL pipeline, which transforms raw Avro logs into a curated Delta table for the data science team, has been sporadically failing over the last two weeks. When the pipeline does not outright fail, its execution time degrades from an expected 45 minutes to over 4 hours. The primary challenge is that the Databricks Jobs UI only surfaces generic `SparkException: Job aborted` errors, leaving the Level 1 support team blind to the actual infrastructure bottleneck causing the degradation.

The underlying compute engine is a 20-node cluster using memory-optimized instances. The infrastructure team suspects that the cluster is suffering from severe Garbage Collection (GC) spirals, ultimately leading to silent OutOfMemory (OOM) driver crashes or executor dropouts. However, because the cluster is ephemeral and terminates after the job runs, the volatile Spark UI and live driver logs are lost before the engineering team can manually inspect them. To prevent further SLA breaches, the architecture board has mandated a programmatic approach to log retention and diagnostic auditing.

As the Senior Reliability Engineer, you are tasked with configuring persistent cluster log delivery to a cloud storage bucket. Once the logs are persisted, you must write an independent diagnostic PySpark script that parses the raw, highly unstructured Log4j `log4j-active.log` and `standard_error` text files generated by the Spark driver and executors. Your script must parse these messy text files to isolate timestamps, extract prolonged GC pause metrics, and identify specific Java heap space memory exceptions, effectively building an automated diagnostic report out of raw infrastructure logs.

Assignment Technical Requirements

To pass this infrastructure diagnostic assessment, your log analysis solution must adhere to the following strict rules:

- **Log Delivery Configuration:** You must configure the Databricks cluster JSON payload to enable `cluster_log_conf`, directing logs to a designated DBFS path (e.g., `dbfs:/cluster-logs/aerocomm-telemetry/`).
- **Unstructured Data Parsing:** You must use PySpark to read the raw text logs as a `DataFrame`. You are strictly forbidden from using standard Python `open()` or `pandas` for reading, as the log volumes are too large for a single machine.

- **Regex Extraction:** The Log4j output is highly unstructured. You must utilize Spark SQL regex functions (`regexp_extract`) to parse the raw string column into structured columns: `Timestamp`, `Log_Level` (e.g., `INFO`, `WARN`, `ERROR`), and `Message`.
- **Metric Isolation:** You must write a downstream transformation that specifically filters for `java.lang.OutOfMemoryError` or messages containing `Pause Young (Allocation Failure)` to isolate the GC bottlenecks.
- **Aggregation:** The final output must aggregate the total number of severe memory errors grouped by `Log_Level` and truncated to the hour, providing a time-series view of the cluster degradation.

7.1 Independent Coding Challenge

Part A: Cluster Log Delivery Configuration

Complete the JSON snippet to enable persistent log delivery for the cluster.

```

1  {
2    "cluster_name": "Telemetry_ETL_Cluster",
3    "spark_version": "13.3.x-scala2.12",
4    "node_type_id": "r5.2xlarge",
5    "num_workers": 20,
6    "cluster_log_conf": {
7      "[TODO: Specify the log destination protocol, e.g., dbfs]": {
8        "destination": "[TODO: Insert the destination path: dbfs:/
9          cluster-logs/aerocomm-telemetry/]"
10     }
11  }
```

Part B: PySpark Log Parsing Script

Complete the PySpark boilerplate to ingest, parse, and aggregate the messy infrastructure logs.

```

1  from pyspark.sql.functions import col, regexp_extract, date_trunc,
    count
2
3  # TODO: Read the raw text logs from the driver's log4j active log
    path
4  # Hint: Use spark.read.text()
5  raw_logs_df = spark.read.[TODO: IMPLEMENT TEXT READER]("dbfs:/
    cluster-logs/aerocomm-telemetry/*/driver/log4j-active.log")
6
7  # The raw log format typically looks like:
8  # "23/10/05 14:32:01 INFO BlockManager: Found block rdd_1_0 locally"
9  # TODO: Use regexp_extract to parse the raw 'value' column into
    structured fields.
10 parsed_logs_df = raw_logs_df.select(
```

```

11     regexp_extract(col("value"), r"^(\\d{2}/\\d{2}/\\d{2} \\d{2}:\\d{2}:\\d{2})", 1).alias("timestamp_string"),
12     regexp_extract(col("value"), r"^\\d{2}/\\d{2}/\\d{2} \\d{2}:\\d{2}:\\d{2} ([A-Z]+)", 1).alias("log_level"),
13     regexp_extract(col("value"), r"^\\d{2}/\\d{2}/\\d{2} \\d{2}:\\d{2}:\\d{2} [A-Z]+ (.*)$", 1).alias("message")
14 )
15
16 # TODO: Cast the timestamp_string to an actual TimestampType.
17 # Note: You may need to use to_timestamp with the specific format '
18 #      yy/MM/dd HH:mm:ss'
19
20 structured_logs_df = parsed_logs_df.withColumn("event_time", [TODO:
21     IMPLEMENT TIMESTAMP CASTING])
22
23 # TODO: Filter the DataFrame to only include rows where the message
24 #      contains "OutOfMemoryError" or "Allocation Failure"
25
26 error_logs_df = structured_logs_df.filter(
27     [TODO: IMPLEMENT STRING MATCHING LOGIC FOR OOM OR GC FAILURES]
28 )
29
30 # TODO: Aggregate the count of errors per hour, grouped by log_level
31
32 diagnostic_report_df = error_logs_df.groupBy(
33     date_trunc("hour", col("event_time")).alias("error_hour"),
34     col("log_level")
35 ).agg(
36     count("*").alias("total_critical_errors")
37 ).orderBy("error_hour")
38
39 display(diagnostic_report_df)

```

Grading Rubric & Solution Verification

Maximum Score: 100 Points (Minimum Passing Score: 85)

- **Log Configuration (15 points):** The `cluster_log_conf` must correctly specify "dbfs" as the key and the "destination" path must be structurally valid.
- **Distributed Text Ingestion (15 points):** Must use `spark.read.text()` (15 points). Using local file I/O operations (`open()`) results in a complete failure of the assignment (0 points).
- **Regex Implementation (30 points):**
 - Correct extraction of the timestamp group (10 points).
 - Correct extraction of the `log_level` group (10 points).
 - Correct extraction of the remaining message payload (10 points).
- **Time Casting & Filtering (25 points):**

- Must correctly map the string pattern 'yy/MM/dd HH:mm:ss' using `to_timestamp()` (10 points).
- The `filter()` must utilize `col("message").like("%OutOfMemoryError%") | col("message").like("%Allocation Failure%")` or `rlike` equivalent (15 points).
- **Aggregation (15 points):** Correct use of `date_trunc("hour", ...)` and grouping logic.

Verification Query Structure

The examiner will inject a synthetically generated 5GB `log4j-active.log` file filled with noise and exactly 412 OOM errors. The candidate's script will execute against this file. The resulting `diagnostic_report_df` must return a precise count of 412 errors distributed across the injected timestamps. Execution time must be under 3 minutes, proving the code leveraged Spark's distributed architecture.

CHAPTER 8

Multi-Task Recovery Protocols & Automated Repair Runs

Scenario-Based Business Problem

GlobalFreight Shipping utilizes a massive Databricks Workflow to process international shipping manifests. The daily DAG consists of 6 sequential and parallel tasks: `Extract_Manifests` → `Validate_Customs` → (`Calculate_Tariffs` & `Update_Inventory`) → `Generate_Invoices` → `Distribute_Reports`. Running the entire workflow from start to finish consumes approximately 40 DBU hours and takes 3.5 hours of wall-clock time. Due to the high compute cost, rerunning the entire job from the beginning in the event of a downstream failure is strictly prohibited by the FinOps steering committee.

Yesterday, the workflow failed at the `Generate_Invoices` task because an external currency conversion API endpoint experienced a 10-minute outage. The upstream tasks (`Extract_Manifests`, `Validate_Customs`, `Calculate_Tariffs`, and `Update_Inventory`) completed successfully and successfully committed terabytes of data to Delta Lake. The Level 1 operations team panicked and manually triggered a completely new run of the entire job. This not only wasted 30 DBU hours but also caused data duplication in the upstream tables because those specific tasks were not programmed to be idempotent.

Your objective as the Lead Data Engineer is to master and implement the Databricks "Repair Run" capability via the Jobs REST API. You must design an automated recovery script that detects a failed Multi-Task Job run, identifies the specific downstream tasks that failed or were skipped, and triggers a specialized Repair Run. This repair run must reuse the successful states of the upstream tasks and strictly re-execute only the failed `Generate_Invoices` task and its subsequent downstream dependency (`Distribute_Reports`).

Assignment Technical Requirements

To pass this recovery protocol assessment, your automated API script must adhere to the following rules:

- **API Protocol:** You must utilize the Databricks REST API 2.1 (`/api/2.1/jobs/runs/repair`) to programmatically heal the pipeline. Manual UI intervention is not allowed.
- **Parameter Retrieval:** The script must accept the `run_id` of the failed parent multi-task job execution.

- **Targeted Execution:** The JSON payload sent to the repair endpoint must explicitly identify the failed task using the `rerun_tasks` array.
- **Dependency Awareness:** The repair payload must successfully orchestrate the resumption of the DAG, ensuring that `Distribute_Reports` runs automatically after the repaired `Generate_Invoices` task succeeds.
- **Idempotency Protection:** The script must include a validation step that queries the job run state (`/api/2.1/jobs/runs/get`) to guarantee it is in a `FAILED` or `CANCELED` state before attempting a repair, preventing accidental concurrent executions.

8.1 Independent Coding Challenge

Using Python and the `requests` library, complete the script to automate the Repair Run protocol. Replace the `[TODO]` placeholders with the precise API logic.

```

1  import requests
2  import json
3  import time
4
5  DATABRICKS_INSTANCE = "https://[TODO: INSERT WORKSPACE URL]"
6  TOKEN = "[TODO: INSERT BEARER TOKEN]"
7  HEADERS = {"Authorization": f"Bearer {TOKEN}", "Content-Type": "
    application/json"}
8
9  def trigger_pipeline_repair(failed_run_id):
10     # Step 1: Verify the run is actually in a failed state
11     get_run_url = f"{DATABRICKS_INSTANCE}/api/2.1/jobs/runs/get?
        run_id={failed_run_id}"
12     response = requests.get(get_run_url, headers=HEADERS)
13     run_details = response.json()
14
15     current_state = run_details.get("state", {}).get("
        life_cycle_state", "")
16     result_state = run_details.get("state", {}).get("result_state",
        "")
17
18     if current_state != "TERMINATED" or result_state != "FAILED":
19         raise Exception(f"Run {failed_run_id} is not in a failed/
            terminated state. Current state: {current_state}")
20
21     print("Run validation passed. Initiating targeted repair...")
22
23     # Step 2: Construct the Repair Run payload
24     repair_url = f"{DATABRICKS_INSTANCE}/api/2.1/jobs/runs/repair"
25
26     # TODO: Construct the JSON payload.
27     # You must target the failed task: "Generate_Invoices".

```

```

28     # Because "Distribute_Reports" depends on it, it will run
        automatically, but you must ensure
29     # the payload strictly identifies the node to restart from.
30     repair_payload = {
31         "run_id": [TODO: INSERT VARIABLE FOR RUN ID],
32         "rerun_tasks": [
33             "[TODO: INSERT THE EXACT FAILED TASK KEY STRINGS HERE]"
34         ]
35     }
36
37     # Step 3: Trigger the repair
38     # TODO: Execute the POST request to the repair_url with the
        repair_payload
39     repair_response = requests.[TODO: HTTP METHOD](repair_url,
        headers=HEADERS, data=json.dumps(repair_payload))
40
41     if repair_response.status_code == 200:
42         new_repair_id = repair_response.json().get("repair_id")
43         print(f"Success! Pipeline repair initiated. Repair ID: {
            new_repair_id}")
44     else:
45         print(f"Failed to trigger repair: {repair_response.text}")
46
47     # Execute the function for a simulated failed run ID
48     trigger_pipeline_repair(failed_run_id=884736291)

```

Grading Rubric & Solution Verification

Maximum Score: 100 Points (Minimum Passing Score: 90)

- **API Endpoint Accuracy (20 points):** Code must correctly construct and target `/api/2.1/jobs/runs/get` and `/api/2.1/jobs/runs/repair` endpoints.
- **State Validation Logic (20 points):** The script must correctly assert that the original `run_id` is both `TERMINATED` and `FAILED` (20 points). Attempting a repair on a `RUNNING` job results in immediate assignment failure.
- **Payload Construction (40 points):**
 - The `run_id` must be correctly mapped as an integer in the payload (10 points).
 - The `rerun_tasks` array must explicitly contain `["Generate_Invoices"]` (30 points). Including downstream tasks like `Distribute_Reports` in this array is redundant but acceptable; however, including upstream tasks like `Extract_Manifests` completely fails the FinOps requirement.
- **HTTP Method Execution (20 points):** Must execute a `requests.post()` for the repair trigger, correctly passing the headers and JSON serialized body.

Verification Query Structure

The examiner will deploy a mock 6-task workflow, intentionally causing Task 4 (**Generate_Invoices**) to throw an exception. The candidate's script will be executed against the workspace. Verification is achieved if a new run executes only tasks 4 and 5, while the cluster initialization time confirms it reused the cached cluster context (if job cluster reuse is active) and bypassed Tasks 1, 2, and 3 entirely.

CHAPTER 9

Enterprise Architecture Capstone Challenge — Part 1

Scenario-Based Business Problem

GlobalPay, a multinational fintech provider, has recently migrated all enterprise data assets into Databricks Unity Catalog. Their architecture is segmented strictly into domains: `catalog_finance`, `catalog_risk`, and `catalog_marketing`. To maintain strict data governance, cross-catalog joins are heavily regulated. The Risk team maintains a critical view named `catalog_risk.fraud_models.high_risk_transactions_vw`. This view relies on underlying tables from both `catalog_risk.core.transactions` and a highly restricted table `catalog_finance.secure.account_balances`.

Yesterday, the daily Risk dashboard failed with a cryptic `[TABLE_OR_VIEW_NOT_FOUND]` or permission-denied error when attempting to query `high_risk_transactions_vw`. The Risk engineers claim they changed nothing. The Finance engineers claim they only applied a routine security patch to their schemas. The result is a broken data lineage chain causing a complete halt in fraud model training. Because multiple teams are involved, pinpointing exactly what broke—whether a column was dropped, a table was renamed, or a `GRANT` was revoked—requires a deep understanding of Unity Catalog’s system lineage and audit tables.

Your capstone objective is to debug this cross-functional data governance crisis. You must write an advanced SQL diagnostic script that queries the Unity Catalog `system.access.table_lineage` to map the exact upstream dependencies of the broken view. Then, you must cross-reference this lineage against the `system.information_schema.table_privileges` to identify exactly which underlying object the `risk_analyst_group` suddenly lost access to. Finally, you must write the exact DCL (Data Control Language) command to restore the broken permissions securely, without over-provisioning access to the Finance catalog.

Assignment Technical Requirements

To resolve this governance crisis, your SQL scripts must adhere to the following strict requirements:

- **Lineage Tracing:** You must query `system.access.table_lineage` to dynamically trace all source tables (`source_table_catalog`, `source_table_schema`, `source_table_name`) that feed into the target view `catalog_risk.fraud_models.high_risk_transactions_vw`.

- **Privilege Auditing:** You must join or use the output of the lineage query against `system.information_schema.table_privileges` to determine if the principal `risk_analyst_group` holds the `SELECT` privilege on all source tables.
- **Root Cause Remediation (DCL):** Assume your diagnostic query discovers that the `SELECT` privilege was accidentally revoked from `catalog_finance.secure.account_balances`. You must write the precise `GRANT` statement to restore this access.
- **Security Constraint:** The restoration grant must provide access only to the `account_balances` table. You must not grant `SELECT` on the entire `catalog_finance.secure` schema.

9.1 Independent Coding Challenge

Write the advanced SQL diagnostic and remediation scripts. Replace the [TODO] placeholders with the exact Unity Catalog system table queries and DCL commands.

```

1  -- Part A: Data Lineage Tracing
2  -- TODO: Query the system table lineage to find all upstream sources
   for the broken view.
3  -- Ensure you filter for the last 7 days of event history to capture
   recent lineage.
4  SELECT
5      source_table_catalog,
6      source_table_schema,
7      source_table_name,
8      entity_type
9  FROM
10     [TODO: INSERT UNITY CATALOG SYSTEM LINEAGE TABLE PATH]
11 WHERE
12     target_table_catalog = 'catalog_risk'
13     AND target_table_schema = 'fraud_models'
14     AND target_table_name = 'high_risk_transactions_vw'
15     AND [TODO: INSERT TIME FILTER FOR THE LAST 7 DAYS];
16
17 -- Part B: Privilege Auditing
18 -- Assume Part A reveals two sources:
19 -- 1. catalog_risk.core.transactions
20 -- 2. catalog_finance.secure.account_balances
21
22 -- TODO: Query the information_schema to verify the current
   privileges held by 'risk_analyst_group'
23 -- on the finance table.
24 SELECT
25     grantee,
26     table_catalog,
27     table_schema,

```

```

28     table_name ,
29     privilege_type
30 FROM
31     [TODO: INSERT THE FINANCE CATALOG INFORMATION_SCHEMA PRIVILEGES
        TABLE]
32 WHERE
33     table_name = 'account_balances'
34     AND table_schema = 'secure'
35     AND grantee = 'risk_analyst_group';
36
37 -- Part C: Precision Governance Remediation
38 -- The audit reveals that 'risk_analyst_group' lost SELECT on
    account_balances.
39 -- TODO: Write the sequence of GRANT commands to restore access.
40 -- Remember the hierarchical requirements of Unity Catalog (Catalog
    -> Schema -> Table).
41 -- Assume 'risk_analyst_group' completely lost all access to the
    finance catalog.
42
43 [TODO: GRANT CATALOG TRAVERSAL COMMAND] ON CATALOG catalog_finance
    TO 'risk_analyst_group';
44 [TODO: GRANT SCHEMA TRAVERSAL COMMAND] ON SCHEMA catalog_finance.
    secure TO 'risk_analyst_group';
45 [TODO: GRANT TABLE READ COMMAND] ON TABLE catalog_finance.secure.
    account_balances TO 'risk_analyst_group';

```

Grading Rubric & Solution Verification

Maximum Score: 100 Points (Minimum Passing Score: 85)

- **System Lineage Query (30 points):**

- Must accurately target `system.access.table_lineage` (15 points).
- Time filter must logically restrict to recent history, e.g., `event_time >= current_timestamp() - INTERVAL 7 DAYS` (15 points).

- **Information Schema Auditing (30 points):** Must correctly target `catalog_finance.information_schema.table_privileges` (30 points). Targeting the global system schema here is also acceptable if scoped correctly (`system.information_schema.table_privileges`).

- **Remediation Hierarchy (40 points):**

- Must explicitly `GRANT USE CATALOG ON CATALOG catalog_finance` (10 points).
- Must explicitly `GRANT USE SCHEMA ON SCHEMA catalog_finance.secure` (10 points).
- Must explicitly `GRANT SELECT ON TABLE catalog_finance.secure.account_balances` (20 points).
- Using `GRANT ALL PRIVILEGES` results in an automatic failure.

Verification Query Structure

The examiner will drop the view's underlying permissions in a sandbox UC environment. The candidate's SQL script will be executed. The validation framework will then switch the execution context to a user within the `risk_analyst_group` and execute a `SELECT COUNT(*)` on `catalog_risk.fraud_models.high_risk_transactions_vw`. A successful count return verifies both lineage deduction and exact permission restoration.

CHAPTER 10

Enterprise Architecture Capstone Challenge — Part 2

Scenario-Based Business Problem

StreamFlix, a global media streaming giant, processes millions of concurrent viewer telemetry events to power their real-time recommendation engine. Their central ETL pipeline, the `Core_Recommendation_Refresh`, is a mission-critical Databricks Multi-Task Workflow. This pipeline operates under a strict SLA: if the recommendations are not refreshed within 4 hours, viewer engagement drops by 15%, equating to millions in lost ad revenue. Recently, the pipeline has been failing silently during weekend spikes. The architecture relies on an overly simplistic DAG: data is ingested, models are trained, and a dashboard is updated. If the training task fails, the entire workflow halts, leaving on-call engineers scrambling to manually intervene at 3:00 AM.

The Vice President of Engineering has mandated the construction of a self-healing, highly resilient workflow architecture. The pipeline must be redesigned to gracefully handle partial failures without requiring human intervention. Specifically, if the `Train_Deep_Learning_Model` task fails, the pipeline should not completely abort. Instead, it should utilize Databricks Workflow's advanced conditional routing to execute a fallback task named `Apply_Heuristic_Model`. Regardless of whether the deep learning model succeeds or the fallback heuristic model is applied, a final `Publish_Recommendations` task must run. Furthermore, any failure in the core path must automatically trigger a webhook alert to the PagerDuty incident response system, complete with system log validation checklists.

As the Principal Architect, your final capstone challenge is to author the complete JSON definition of this fault-tolerant production pipeline. You must integrate `Run If` condition tasks to handle the branching logic between the primary machine learning path and the heuristic fallback path. You must also implement Job-level webhook notifications to satisfy the incident response requirements, ensuring that complex dependencies are deeply understood and mapped for a true enterprise-grade layout.

Assignment Technical Requirements

To pass this final capstone assessment, your Workflow JSON payload must adhere to the following advanced orchestration rules:

- **Conditional Branching (Run If):** You must configure task dependencies such that `Apply_Heuristic_Model` only executes if the upstream `Train_Deep_Learning_Model` task fails (`run_if`: `"AT_LEAST_ONE_FAILED"` or similar failure condition).

- **Convergence Task:** The `Publish_Recommendations` task must execute after either the deep learning model succeeds or the heuristic model completes. You must configure its `run_if` condition to allow execution even if one of its upstream branches was skipped (e.g., `run_if: "ALL_DONE"` or specific task-based custom conditions).
- **Webhook Integration:** You must define a `webhook_notifications` block at the Job level. It must trigger on job `on_failure` and send an alert to an existing PagerDuty integration endpoint ID.
- **Compute Economy:** As established in Chapter 4, the entire workflow must utilize a single shared `job_cluster` to minimize node bootstrapping time.
- **Log Validation:** A final task, `Validate_System_Logs`, must run only if the deep learning model succeeds, running a simple SQL assert query to check row counts.

10.1 Independent Coding Challenge

Complete the hyper-exhaustive JSON representation of the `Core_Recommendation_Refresh` workflow. Replace the [TODO] placeholders to implement the advanced branching, convergence, and notification logic.

```

1  {
2    "name": "Core_Recommendation_Refresh_Resilient",
3    "webhook_notifications": {
4      "on_failure": [
5        {
6          "id": "[TODO: INSERT PAGERDUTY WEBHOOK ID: e.g., 'pd-webhook
7            -998']"
8        }
9      ],
10   "job_clusters": [
11     {
12       "job_cluster_key": "resilient_compute_engine",
13       "new_cluster": {
14         "spark_version": "13.3.x-scala2.12",
15         "node_type_id": "r5.8xlarge",
16         "num_workers": 12
17       }
18     }
19   ],
20   "tasks": [
21     {
22       "task_key": "Ingest_Telemetry",
23       "job_cluster_key": "resilient_compute_engine",
24       "notebook_task": {
25         "notebook_path": "/ML/01_Ingest"
26       }
27     },

```

```

28     {
29         "task_key": "Train_Deep_Learning_Model",
30         "depends_on": [{"task_key": "Ingest_Telemetry"}],
31         "job_cluster_key": "resilient_compute_engine",
32         "notebook_task": {
33             "notebook_path": "/ML/02_Train_DL"
34         }
35     },
36     {
37         "task_key": "Apply_Heuristic_Model",
38         "depends_on": [{"task_key": "Train_Deep_Learning_Model"}],
39         "job_cluster_key": "resilient_compute_engine",
40         "notebook_task": {
41             "notebook_path": "/ML/02b_Fallback_Heuristic"
42         },
43         "run_if": "[TODO: INSERT CONDITION FOR WHEN UPSTREAM FAILS]"
44     },
45     {
46         "task_key": "Publish_Recommendations",
47         "depends_on": [
48             {"task_key": "Train_Deep_Learning_Model"},
49             {"task_key": "Apply_Heuristic_Model"}
50         ],
51         "job_cluster_key": "resilient_compute_engine",
52         "notebook_task": {
53             "notebook_path": "/ML/03_Publish"
54         },
55         "run_if": "[TODO: INSERT CONDITION SO IT RUNS WHEN EITHER
                    UPSTREAM PATH FINISHES]"
56     },
57     {
58         "task_key": "Validate_System_Logs",
59         "depends_on": [{"task_key": "Train_Deep_Learning_Model"}],
60         "job_cluster_key": "resilient_compute_engine",
61         "sql_task": {
62             "query": {"query_id": "assert-logs-001"}
63         },
64         "run_if": "[TODO: INSERT CONDITION SO IT ONLY RUNS IF DL MODEL
                    SUCCEEDS]"
65     }
66 ]
67 }

```

Grading Rubric & Solution Verification

Maximum Score: 100 Points (Minimum Passing Score: 90)

- **Failure Branching Implementation (30 points):** The `Apply_Heuristic_Model` task must explicitly possess `"run_if": "ALL_FAILED"` or `"AT_LEAST_ONE_FAILED"` mapping back to its dependency on `Train_Deep_Learning_Model`.

- **Convergence Execution Logic (30 points):** The `Publish_Recommendations` task depends on two tasks, one of which will inevitably be skipped or failed. It must possess `"run_if": "NONE_FAILED"` or `"ALL_DONE"` depending on exact Databricks API versioning, or utilize the `condition_task` syntax if mimicking standard `ALL_DONE` semantics. (Acceptable answer: `"NONE_FAILED"` assuming the skipped heuristic task is evaluated as a success, or explicitly `"ALL_DONE"`).
- **Success-Only Pathing (20 points):** `Validate_System_Logs` must explicitly use `"run_if": "ALL_SUCCESS"` to ensure it only executes on the primary happy path.
- **Alerting Integration (20 points):** The `webhook_notifications` block must correctly nest the endpoint ID under the `on_failure` array.

Verification Query Structure

The examiner will deploy this JSON payload into a production environment.

1. **Test 1 (Happy Path):** `Train_Deep_Learning_Model` is forced to succeed. The engine must verify that `Apply_Heuristic_Model` is `SKIPPED`, `Validate_System_Logs` `SUCCEEDS`, and `Publish_Recommendations` `SUCCEEDS`.
2. **Test 2 (Disaster Path):** `Train_Deep_Learning_Model` is forced to fail via an injected exception. The engine must verify that `Apply_Heuristic_Model` `RUNS AND SUCCEEDS`, `Validate_System_Logs` is `SKIPPED`, `Publish_Recommendations` `RUNS AND SUCCEEDS`, and a PagerDuty API POST request is recorded in the network logs.